

Récap session — Recherche & affichage des praticiens

Projet Abi · Mai 2026

1. Réflexions UX — Architecture de la recherche

Question de départ

Faut-il afficher les praticiens directement sur la homepage, ou ouvrir une nouvelle page après la recherche ?

Décision

Page de résultats séparée (/search) — c'est la bonne architecture.

Pourquoi :

- La homepage reste épurée, centrée sur la promesse (le Hero + la search bar)
- Les résultats ont besoin d'espace : cards, filtres, carte géographique
- Pattern connu des utilisateurs (Doctolib, Airbnb, Google Maps) → pas de friction
- Les URLs deviennent partageables : `/search?specialty=osteo&city=Lyon`
- La homepage reste légère et rapide à charger

Quand afficher les cards sur la homepage ? Uniquement si tu as très peu de contenu (< 10 praticiens) pour donner une impression de vie — solution de démarrage, pas scalable.

2. Réflexions UX — Affichage des praticiens

Deux propositions analysées

Option A — Liste + Carte (split view)

- Liste scrollable à gauche, carte interactive à droite
- Épingles sur la carte = correspondances dans la liste
- Pattern Doctolib / Airbnb
- **Pour** : puissant, dimension géographique visuelle, très différenciant
- **Contre** : nécessite une lib de carte (Leaflet/Mapbox) + coordonnées GPS en base

Option B — Grille de cards (simple)

- Grid responsive 1→2→3 colonnes
- **Pour** : rapide à développer, mobile-first
- **Contre** : pas de dimension géographique

Décision

Option A retenue — avec placeholder carte pour la v1, la carte réelle en v2.

3. Réflexions UX — Design de la card praticien

Deux propositions de card

Proposition 1 — Card compacte "scan rapide"

- Info condensée, tags en ligne
- Badge "validé asso" en haut à droite, discret

Proposition 2 — Card enrichie "confiance" retenue

- Avatar initiales + nom + spécialité + ville + prix
- **Bandeau dédié** "♦ Validé par : [Nom de l'asso]" en **bg-peach**
- Tags officiels groupés par catégorie (Pathologie / Inclusivité / Pratique)
- Tags communauté avec compteur de votes
- Bouton sauvegarder + lien profil

Pourquoi la Prop 2 est plus forte : Afficher le nom de l'asso qui valide est un signal de sérieux et de transparence — c'est la valeur ajoutée d'Abi par rapport à un simple annuaire.

4. Bonnes pratiques techniques

Architecture des fichiers

Fichier	Rôle
<code>lib/practitioners-search.ts</code>	Requêtes DB pour la liste + suggestions
<code>lib/practitioners-detail.ts</code>	Requête DB pour la fiche individuelle
<code>components/UI/search/PractitionerCard.tsx</code>	Card de résultat (client component)
<code>components/UI/search/SearchCombobox.tsx</code>	Input avec suggestions (client component)
<code>app/search/page.tsx</code>	Page résultats (server component)
<code>app/praticiens/[id]/page.tsx</code>	Fiche individuelle (server component)

Pourquoi séparer **practitioners-search** et **practitioners-detail** ?

- Séparation des responsabilités : chaque fichier a un périmètre clair
- Plus facile à maintenir : si la recherche évolue, on ne touche pas à la fiche
- Un dev junior peut retrouver immédiatement où est la logique qu'il cherche

Server Components vs Client Components

Server Component

- Fetch DB directement

Client Component

- Interactions utilisateur (useState)

- Pas de JS envoyé au navigateur
- Sauvegarder un praticien
- Hero, SearchPage, FichePage
- SearchCombobox, PractitionerCard

Règle pratique : tout ce qui a besoin de `useState` ou d'un event listener (`onClick`, `onChange`) doit être `"use client"`.

Données masquées selon l'authentification

Adresse et téléphone du praticien sont masqués pour les non-connectés (commenté dans le schéma DB). Sur la fiche individuelle :

```
// On vérifie la session côté serveur
const session = await auth.api.getSession({ headers: await headers() });
const isLoggedIn = !!session;

// Puis dans le JSX :
{isLoggedIn ? (
  <p>{address}</p>
) : (
  <p>🔒 Connectez-vous pour voir les coordonnées</p>
)}
```

5. L'autocomplete — SearchCombobox

Question posée

Comment faire "taper intuitivement" la spécialité ou la ville ?

Trois options comparées

Option	Complexité	Style	Description
<datalist> HTML natif	★	Navigateur	5 lignes, pas stylisable
Combobox simple	★★	Custom Abi	<code>useState</code> + filtre
Combobox complet	★★★	Custom Abi	+ navigation clavier, <code>useRef</code>

Décision : Combobox simple ✅

Logique du composant :

1. Les suggestions viennent de la DB (fetchées côté serveur, passées en props)
2. L'utilisateur tape → `useState` filtre la liste en temps réel
3. `onFocus` → ouvre la liste

4. `onBlur` (avec 150ms de délai) → ferme la liste
5. `onMouseDown` → copie la suggestion dans l'input + ferme

Pourquoi le délai de 150ms sur `onBlur` ? Sans lui, le `onBlur` se déclenche avant le `onClick` de la suggestion → la liste se ferme avant que la valeur soit copiée. Le délai laisse le temps au clic de s'exécuter.

Pourquoi `value.length >= 1` pour afficher la liste ? Si on affiche la liste dès le `onFocus`, toutes les suggestions s'ouvrent sur un champ vide. Avec `>= 1`, la liste n'apparaît qu'une fois que l'utilisateur commence à taper.

```
[ spécialité ]    ← focus seul → rien
[ o ]            ← tape "o" → Gynécologue, Ostéopathe, Sophrologue
[ os ]           ← tape "os" → Ostéopathe, Sophrologue
```

6. ESLint — Bonnes pratiques

Erreur rencontrée

```
Error: `` can be escaped with `&apos;`  react/no-unescaped-entities
```

Cause : En JSX, les apostrophes dans le texte doivent être échappées car `'` est un caractère réservé HTML.

```
// ❌ Interdit en JSX
<p>Connectez-vous pour voir l'adresse</p>

// ✅ Correct
<p>Connectez-vous pour voir l&apos;adresse</p>
```

Warnings — Variables inutilisées

```
Warning: 'setMode' is assigned a value but never used
Warning: 'isLoading' is assigned a value but never used
```

Convention `_` pour les variables intentionnellement inutilisées :

```
// ❌ Déclenche le warning
const [mode, setMode] = useState("signup");
const [isLoading, setIsLoading] = useState(false);
```

```
// ✅ Le préfixe _ indique "intentionnellement inutilisé"
const [mode, _setMode] = useState("signup");
const [_isLoading, setIsLoading] = useState(false);
```

Ce que ça communique : "Je sais que cette variable n'est pas utilisée maintenant, c'est voulu — elle sera implémentée plus tard."

7. Schéma de données — ce qui pilote l'UI

```
practitioners          → nom, spécialité, ville, prix, convention
practitionerTags       → tags attitrés (officiels)
tags                   → label + category (pathologie / inclusivité /
pratique)
tagVotes               → tags ajoutés/votés par la communauté (avec
compteur)
practitionerAssociations (validationStatus = "approved")
                      → badge "+ Validé par [Nom asso]"
associations           → nom + site web de l'asso
```

Règle d'affichage :

Source	Affichage card	Affichage fiche
<code>practitionerTags</code>	Pills par catégorie	Sections par catégorie
<code>tagVotes</code>	👍 tag xN	👍 tag xN
<code>practitionerAssociations</code>	Bandeau peach	Bandeau peach + lien site
<code>address / phone</code>	Non affiché	Affiché si connecté, sinon 🗝

8. Résumé des décisions prises

Sujet	Décision
Architecture recherche	Page <code>/search</code> séparée
Affichage résultats	Split view liste + carte (placeholder v1)
Design card	Proposition 2 — card enrichie "confiance"
Autocomplete	Combobox simple (useState + filtre)
Fichiers lib	Séparés : <code>practitioners-search.ts</code> / <code>practitioners-detail.ts</code>
Données sensibles	Adresse/téléphone masqués si non connecté

Document généré à partir de la session de travail du 27 mai 2026.